

Model Selection

To predict IMDb movie ratings, we evaluated three machine learning models—Linear Regression, Artificial Neural Network (ANN), and Random Forest (RF)—chosen for their complementary strengths in prior studies. Linear Regression was selected for its interpretability, as shown by [1] and [2] in predicting ratings using movie metadata. ANN was included for its ability to model non-linear relationships, validated by [4], while Random Forest was chosen for its robustness to overfitting, supported by [6]. These models enable a comprehensive comparison of linear, non-linear, and ensemble approaches.

Model Definitions

Linear regression is a statistical method that models a linear relationship between a dependent variable and one or more independent variables, where prediction is a weighted sum of the input features plus an intercept, optimized by minimizing the mean squared error (MSE).

An Artificial Neural Network (ANN) is a machine learning technique that learns complex patterns through interconnected layers of nodes (neurons) that process input features through weighted connections, applying activation functions to capture non-linear relationships.

Random Forest (RF) is a machine learning algorithm that creates multiple independent decision trees based on a random subset of data and features, and predicts by averaging the predictions of all trees.

Performance Evaluation

Model performance was assessed using mean squared error (MSE), root mean squared error (RMSE), and R-squared (R^2), standard metrics for regression tasks [1], [2]. MSE measures the average squared difference between predicted and actual ratings, emphasizing larger errors and providing a precise measure of prediction accuracy. RMSE, the square root of MSE, is expressed in the original rating scale, making it intuitive for comparing model performance. R^2 indicates the variance in the ratings explained by the model. These metrics were selected to ensure a comprehensive evaluation of model accuracy, error magnitude, and goodness of fit.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad RMSE = \sqrt{MSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Shared Methodology

For each of the three models created, features were standardized using StandardScaler to ensure comparable scales. The methodology leveraged Python libraries: pandas for data handling, numpy for computations, scikit-learn for preprocessing (StandardScaler, train_test_split, GridSearchCV), modeling (LinearRegression, RandomForestRegressor), and evaluation (mean_squared_error, r2_score), tensorflow and keras for ANN implementation (Sequential, Dense, SGD), and scikeras for Keras integration. Model performance was assessed using R^2 , MSE, and RMSE, with results saved as CSV files.

Linear Regression

A linear regression model was trained on standardized features using 5-fold cross-validation, with shuffling to reduce bias and mitigate overfitting, as is standard in regression tasks [1]. The model was trained on each fold, with performance measured using R^2 , MSE, and RMSE. Average feature coefficients were calculated across folds to interpret feature importance.

Artificial Neural Network (ANN)

An ANN was developed with a single hidden layer, standardized features, and hyperparameter tuning (neurons, activation, momentum, batch size) was performed via 3-fold cross-validation with grid search, using early stopping to prevent overfitting. The hyperparameter grid for the Artificial Neural Network (ANN) model was chosen to optimize performance while balancing computational efficiency. Neuron counts (32, 64, 128) were selected to test a range of network capacities, from simple to complex. Activation functions (tanh, ReLU) were included to address the handling of non-linear patterns, as supported by prior studies [4]. Momentum values (0.6, 0.9) and batch sizes (32, 64) were selected to investigate the trade-offs between training stability and convergence speed. The best model was trained on a randomly selected 80/20 train-test split and evaluated using R^2 , MSE, and RMSE.

Random Forest (RF)

To predict IMDb movie ratings, we implemented a Random Forest Regressor. Features were standardized, and an 80/20 train-test split was applied. Hyperparameter tuning ($n_estimators$, max_depth , and $min_samples_split$) was performed using 3-fold cross-validation with a grid search to optimize performance. The hyperparameter grid was selected to find a combination of hyperparameters that balances model complexity and performance. The best model was evaluated using R^2 , MSE, and RMSE. The number of trees ($n_estimators$: 100, 200) was chosen to achieve accurate predictions with acceptable computational cost. Maximum tree depth (max_depth : 10, 20, None) and minimum samples per split ($min_samples_split$: 2, 5) were included to explore the trade-offs between overfitting and generalization.

References

- [1] P.-Y. Hsu, Y.-H. Shen, and X.-A. Xie, "Predicting movies user ratings with Imdb attributes," *Lecture Notes in Computer Science*, pp. 444–453, 2014. doi:10.1007/978-3-319-11740-9_41
- [2] H. Chamani, Z. S. Zadeh, and B. Bahrak, "An overview of regression methods in early prediction of movie ratings," *2021 11th International Conference on Computer Engineering and Knowledge (ICCKE)*, pp. 1–6, Oct. 2021. doi:10.1109/iccke54056.2021.9721453
- [4] W. R. Bristi, Z. Zaman, and N. Sultana, "Predicting imdb rating of movies by Machine Learning Techniques," *2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, Jul. 2019. doi:10.1109/icccnt45670.2019.8944604
- [6] S. M. Abidi, Y. Xu, J. Ni, X. Wang, and W. Zhang, "Popularity prediction of movies: From Statistical Modeling to Machine Learning Techniques," *Multimedia Tools and Applications*, vol. 79, no. 47–48, pp. 35583–35617, Jan. 2020. doi:10.1007/s11042-019-08546-5

